DENIS

User Manual

Doppler Estimation and Numerical Inference for Spectroscopy

Version 1.1.1

Arda Kayaalp
arda.kayaalp@kuleuven.be

October 20, 2026

Contents

1 Introduction **4**

2 Installation **5**
2.1 Recommended: scripted install . . . 5
2.2 Manual install . . . 5
2.3 Editor / IDE integration . . . 6
2.4 Updating . . . 6
2.5 Troubleshooting . . . 6

3 Quick Start **6**
3.1 Launching the application . . . 6
3.2 Setting up an estimation . . . 7
3.3 Pre-analysing measured data . . . 7
3.4 Building an analysis project . . . 8
3.5 Running the fit and reading the result . . . 11
3.6 Combining isotopes (isotope shifts) . . . 11

4 Estimate Tab **11**
4.1 Global parameters . . . 12
4.2 Isotope entries . . . 12
4.3 Running the estimation . . . 13
4.4 Simulated spectra . . . 13
4.5 Loading a previous estimation . . . 14

5 Pre-Analysis Tab **14**
5.1 Loading data . . . 14
5.2 The time-of-flight and timestamp gates . . . 14
5.3 Display axis and physics inputs . . . 15
5.4 Binning . . . 15
5.5 HFS model overlay . . . 15
5.6 Per-scan filtering . . . 16
5.7 Virtual splits . . . 16
5.8 Merging runs . . . 16
5.9 Calibrations sub-tab . . . 16
5.10 Cooler Voltage sub-tab . . . 16
5.11 Per-run voltage calibration . . . 16

6 Analysis Tab **17**
6.1 Projects and the block pipeline . . . 17
6.2 Source block . . . 17
6.3 Model block . . . 18
6.4 Fitter block . . . 19
6.5 Output block . . . 20
6.6 Auto-Fitter . . . 20
6.7 Isotope Shifts / Centroid Correction sub-tab . . . 21
6.8 Reference correction (Gaussian process) . . . 21

7 Results Tab **22**
7.1 The plot editor . . . 22
7.2 Exporting . . . 22
7.3 Refresh, collapse, and deletion . . . 23

8 Tools Menu **23**
8.1 Schmidt Moment Calculator . . . 23
8.2 Shell Configuration Plotter . . . 24
8.3 Unit Converter . . . 24
8.4 SHG Crystal Angle Calculator . . . 24
8.5 SFG / DFG Calculator . . . 25
8.6 Quick Plot . . . 25
8.7 ASDF Viewer . . . 25
8.8 NIST ASD Browser . . . 26

9 Settings and Save / Load **27**
9.1 Settings dialog . . . 27
9.2 Save and Load . . . 28

A Physics reference **29**
A.1 Doppler shift . . . 29
A.2 Hyperfine structure . . . 29
A.3 Schmidt magnetic moments . . . 29
A.4 Fit objectives . . . 30

B Troubleshooting **30**

C Citation and license **31**
C.1 Citing DENIS . . . 31
C.2 License . . . 31
C.3 Contact . . . 31

References **32**

1 Introduction

DENIS (*Doppler Estimation and Numerical Inference for Spectroscopy*) is an interactive desktop application for planning, analysing, and reporting collinear laser spectroscopy (CLS) measurements.

In a CLS experiment, an ion or atom beam is overlapped collinearly (or anti-collinearly) with a frequency-narrow laser, and the hyperfine structure (HFS) of an electronic transition is mapped by scanning the beam acceleration voltage to Doppler-tune the laser into resonance with each isotope. Quantitative analysis of the resulting spectra yields isotope shifts, magnetic dipole and electric quadrupole moments, and spin assignments — the primary observables of nuclear structure measurements at facilities such as IGISOL, ISOLDE, and FRIB. For background see, e.g., the Handbook of Nuclear Physics chapter on nuclear spins and moments [1].

DENIS brings the full experiment cycle into a single tool, organised as four top-level tabs:

Estimate. Predicts beam time. Given an element, transition, isotope chain (with masses, spins, magnetic moments and quadrupole moments), beam parameters, and yields, it computes per-peak signal rates, the time required to reach a target statistical uncertainty, and a total run-time budget.

Pre-Analysis. Loads experiment data files (IGISOL-style ASDF via `clstools` [2]) and lets the user inspect the time-of-flight gate, set the binning axis (acceleration voltage or Doppler-shifted rest-frame frequency), and overlay a trial HFS model to seed initial fit parameters.

Analysis. Performs the actual fitting. Each analysis project chains a **Source**, one or more **Model** blocks, a **Fitter**, and one or more **Output** blocks. Multi-source and multi-model fits share parameters across runs. An **Auto-Fitter** explores the parameter space with a multi-start parallel `leastsq` sweep when the human seed is too far from the global minimum. Fitting itself is powered by SATLAS2 [3].

Results. Collects per-project fit summaries, exposes the matplotlib figures for editing (line widths, fonts, colours, axis limits) with persistent style files, and exports plots in PNG, PDF, or SVG. A permanent **Isotope Shifts** sub-tab combines fit centroids across projects to compute $\delta\nu$ relative to a chosen reference isotope.

A Tools menu provides supporting utilities (Schmidt moment calculator, shell configuration plotter, unit converter, SHG crystal angle calculator, SFG / DFG calculator, quick plot, and a NIST Atomic Spectra Database browser with a laser excitation-scheme finder) for tasks adjacent to the main workflow; see Section 8.

Supported data format. The **Pre-Analysis** and **Analysis** tabs in the current release read *only* IGISOL-style ASDF files, via the `clstools` [2] reader. Other facility formats are not supported out of the box; adapting DENIS to a different layout requires editing the source-block loader. The **Estimate** tab has no file-format dependency and can be used independently of any data on disk.

Audience. This manual assumes the reader has hands-on experience running CLS measurements and is comfortable with concepts such as Doppler tuning, hyperfine structure, the A and B coupling constants, and Schmidt moments. For background on the underlying physics see References [1, 3] and Appendix A.

Scope. This is a user manual, not an API reference: it documents the GUI workflow and supporting concepts, not the internal Python modules. The backend code (in `cls_estimations/` and `gui/`) is intended to be read directly by developers extending DENIS.

2 Installation

DENIS runs on Windows, macOS, and Linux. The recommended install flow uses uv (a fast Python package and project manager from Astral) to manage Python itself, the project virtual environment, and all dependencies in one step. No existing Python installation is required: uv downloads a compatible interpreter on first sync.

2.1 Recommended: scripted install

After cloning the repository, open a terminal in the project root and run the platform-specific install script.

```
# Windows (cmd or PowerShell)
install.bat

# Linux / macOS
./install.sh
```

The script:

1. Checks whether uv is on PATH; if not, prompts to install it via the official Astral installer.
2. Runs uv sync, which downloads a compatible Python interpreter (if needed), creates .venv/ in the project root, and installs every dependency pinned in uv.lock.
3. Creates a desktop shortcut (also available on its own via install.bat shortcut / ./install.sh shortcut): a Windows .lnk pointing at .venv\Scripts\pythonw.exe, a Linux .desktop entry, or a clickable .command on macOS.

When the script finishes, double-click the desktop shortcut (Figure 1) to launch DENIS, or run from the terminal:

```
uv run python gui.py
```

The first launch opens a splash screen, then the main window with the four tabs introduced in Section 1.



Figure 1: The DENIS desktop shortcut created on Windows by the installer. The Linux .desktop entry and macOS .command launcher use the same icon.

2.2 Manual install

If uv is already installed and the wrapper scripts are not needed:

```
git clone https://github.com/ardakayaalp/DENIS.git
cd DENIS
uv sync
uv run python gui.py
```

The `clstools` [2] dependency is not published on PyPI and is fetched from GitHub via the `[tool.uv.sources]` block in `pyproject.toml`. `uv sync` handles this automatically; no manual step is needed.

2.3 Editor / IDE integration

For VSCode or PyCharm, point the project Python interpreter at the `uv`-managed environment so debugging and linting use the same packages as the running app:

- Windows: `.venv\Scripts\python.exe`
- Linux / macOS: `.venv/bin/python`

2.4 Updating

To pull a newer version of DENIS and refresh dependencies:

```
git pull
uv sync
```

`uv sync` reads `uv.lock` and brings the local environment to exactly that pinned set — adding, removing, and upgrading packages as required.

2.5 Troubleshooting

`uv` installed but not found. The Astral installer drops `uv` into `~/.local/bin` (Linux/macOS) or `%USERPROFILE%\.local\bin` (Windows). Open a new terminal so the updated `PATH` takes effect, or add the directory to `PATH` manually.

`clstools` fetch fails. `uv sync` clones `cls_tools` from GitHub. A failure usually means the machine has no GitHub access. Clone the repository to a sibling folder manually and replace the entry in `[tool.uv.sources]` with a local path:

```
[tool.uv.sources]
clstools = { path = "../cls_tools" }
```

Desktop shortcut opens a console on Windows. `create_shortcut.bat` prefers `.venv\Scripts\pythonw.ex` if it falls back to `python.exe`, `pythonw.exe` is missing from the environment. Re-run `uv sync` to repair.

macOS `.command` launcher will not open. macOS may quarantine downloaded scripts. Right-click the `DENIS.command` file, choose **Open**, and allow execution at the Gatekeeper prompt; or grant execute permission with `chmod +x ~/Desktop/DENIS.command`.

3 Quick Start

This walkthrough takes the reader from a fresh install through one complete cycle: estimate beam time, pre-analyse a measured spectrum, fit it, and review the result. No example data is bundled with this release — substitute paths to your own ASDF file where indicated.

3.1 Launching the application

Double-click the desktop shortcut created during install (Section 2), or run `uv run python gui.py` from the project root. After a brief splash the main window opens with the four tabs across the top: **Estimate**, **Pre-Analysis**, **Analysis**, **Results**.

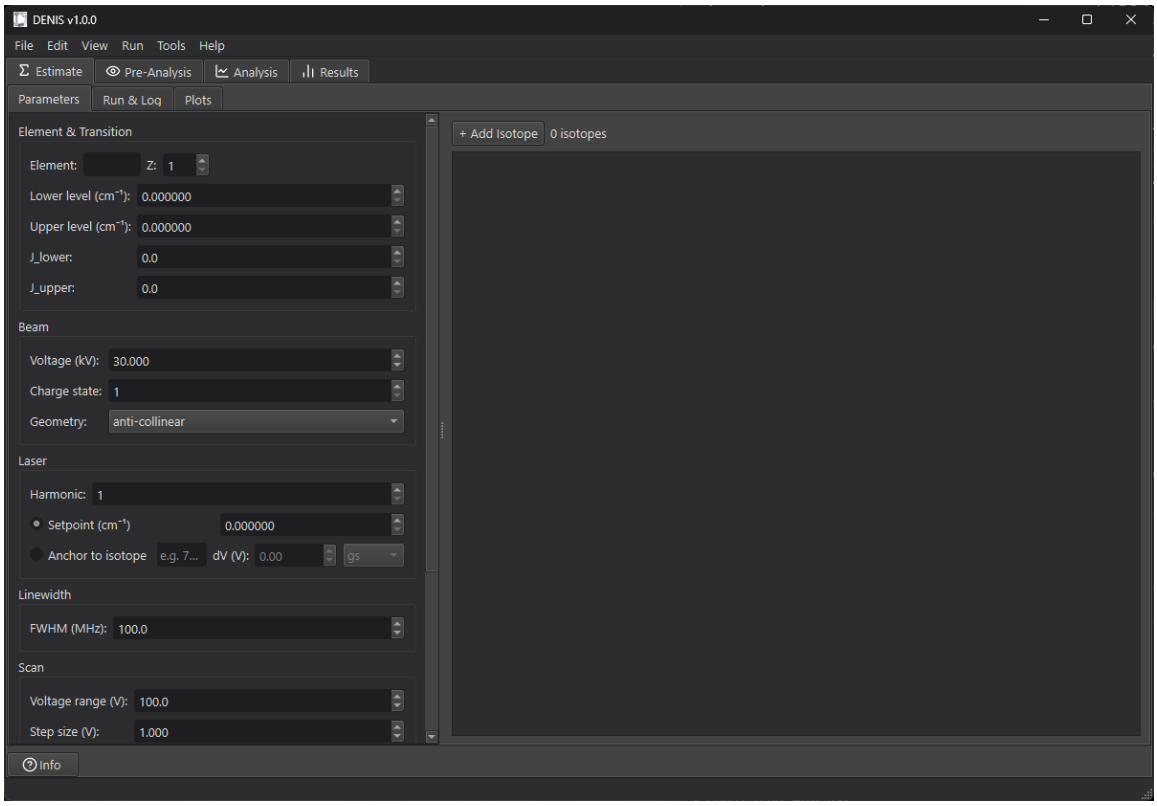


Figure 2: The DENIS main window after launch, with the four top-level tabs visible. The **Estimate** tab is opened by default; its isotope column is empty until isotopes are added.

3.2 Setting up an estimation

Open the **Estimate** tab. The left column of global parameters holds the element symbol and atomic number, the lower and upper levels of the electronic transition (in cm^{-1}) with their total angular momenta J, the laser harmonic, the beam acceleration voltage, and the scan range, step size, and dwell time.

The middle column contains one panel per isotope (**+ Add Isotope**). For each isotope, supply the mass number A, nuclear spin I, magnetic moment μ (in nuclear magnetons), quadrupole moment Q (in barns), expected isotope shift, ion yield (ions/s), and spectroscopic efficiency. Mark one isotope as the **Reference** — its hyperfine A and B constants (entered in MHz) are used to scale the constants of the others via nuclear-moment ratios.

When the form is complete, click **Run Estimation** (right column). The calculation streams into the log at the bottom of that column: per-peak signal rates, the on-peak time needed for the requested σ, the number of sweeps to cover the chosen voltage range, and a grand-total run time. The plot panel above the log shows the simulated spectrum for each isotope on the requested scan range; its **View** drop-down includes a sortable **Peak List** of every hyperfine component, and the log prints the peaks sorted both by intensity and by voltage.

3.3 Pre-analysing measured data

Switch to the **Pre-Analysis** tab and click **Open...**. Select your IGISOL-style ASDF run file (for example `~/data/your_run.asdf`). The file appears in the file list on the left with its checkbox ticked, and its time-of-flight and voltage spectra load into the centre panels.

Enable the TOF **Gate** and drag the orange region (or type the bounds) to isolate the ion bunch from background. After clicking inside a model value box, the keyboard arrows step the value by the digit under the cursor — click on a coarse digit for big jumps, on a fine one for

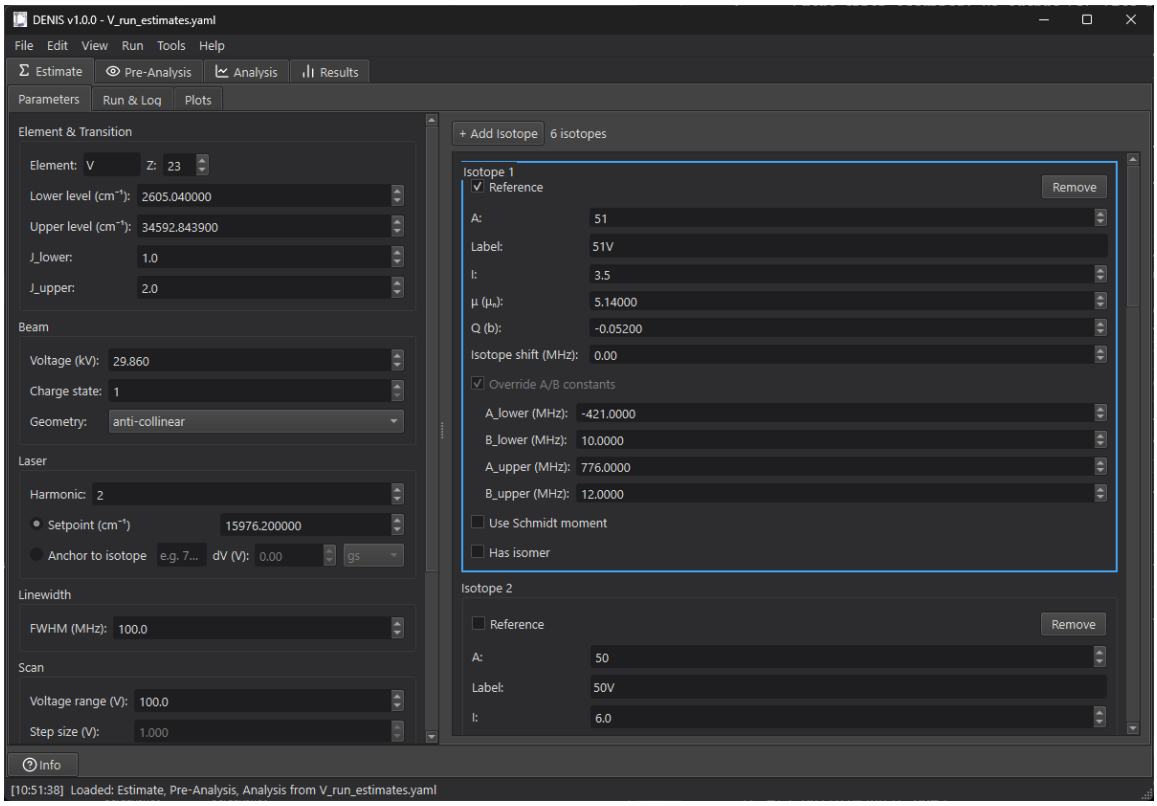


Figure 3: Estimate tab with the global parameters (**Element & Transition**, **Beam**, **Laser**, **Linewidth**, **Scan**) filled in for a vanadium chain, and the first isotope panel (^{51}V, marked **Reference**) expanded to show the per-isotope inputs and an explicit A/B override.

small adjustments. The voltage spectrum re-bins live as the gate moves.

To inspect the spectrum in the rest frame, switch the **X-axis** from **Voltage** to **Frequency** (or **Wavenumber**). To overlay a trial HFS model, click **+ Add Model** in the left column and set the centroid, linewidths, and A / B constants; the model curve updates live on top of the data. Iterate until the trial line positions sit close to the visible peaks — this gives a clean seed for the fitter.

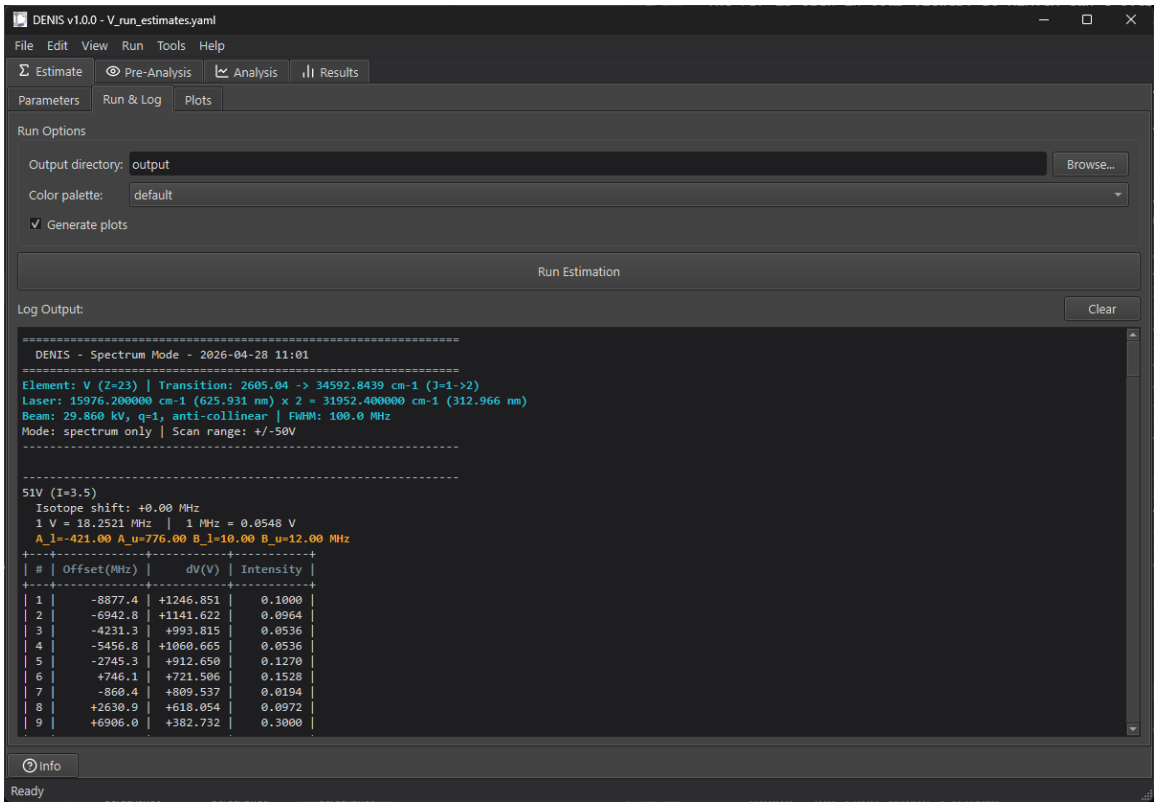
3.4 Building an analysis project

Open the **Analysis** tab and click **+ Create Analysis Project**. Each project lives in its own sub-tab named after the isotope or measurement.

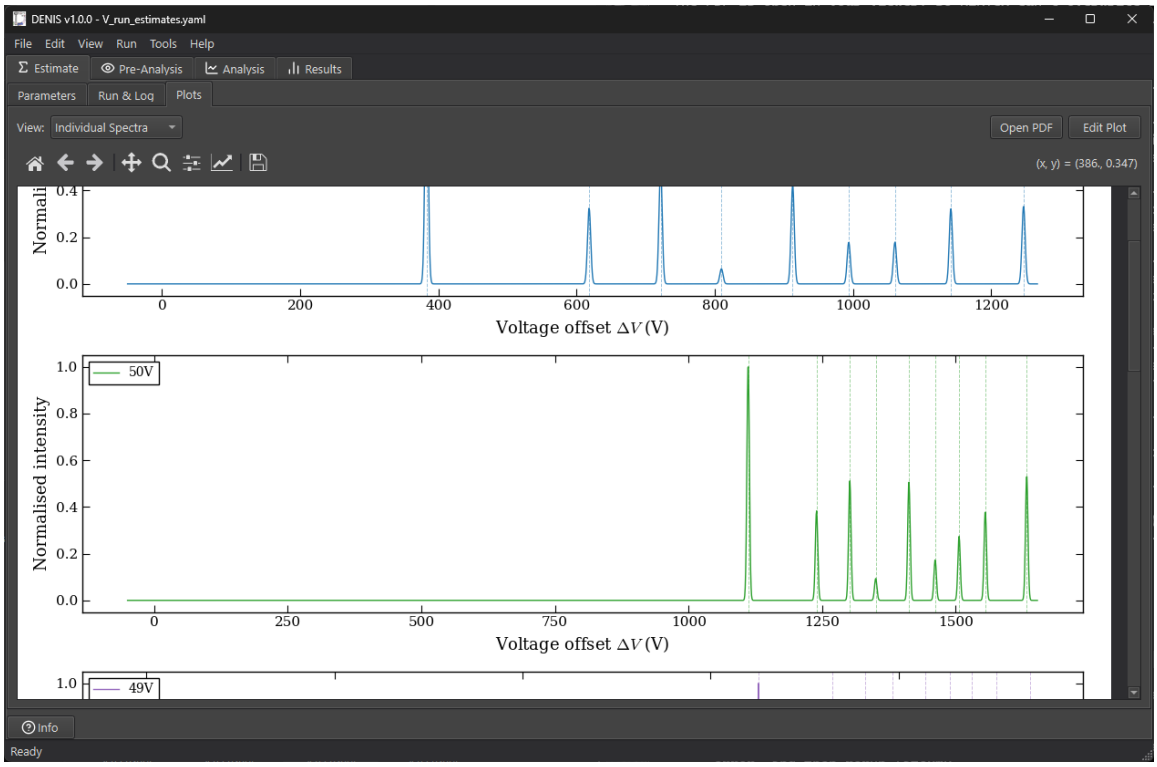
Inside the project, assemble a pipeline of blocks (drag the handle on the left of each block to reorder):

1. A **Source** block holding your run files (use **Import from Pre-Analysis** to pull the files, physics parameters, and gates you settled on there in one click).
2. One or more **Model** blocks — typically one per isotopomer (ground state, isomer). Set initial A, B, amplitudes, FWHMs, and centroid, and set the **Mode** of the parameters that should float to **Free**.
3. One **Fitter** block — pick the optimiser (`leastsq`, `least_squares`, `slsqp`, `emcee`, ...) and the statistic (chi-square / Poisson LLH / Gaussian LLH).
4. One or more **Output** blocks — enable the plots you want (data + fit, residuals, walk plot for MCMC, parameter correlations, χ^2 map).

If the initial parameter guess is too far from the global minimum, click **Find Parameters** on the **Fitter** block. The Auto-Fitter runs a multi-start parallel `leastsq` sweep, lets you compare



(a) The run log. The header summarises the inputs; the body lists each hyperfine peak with offset, voltage, and intensity.



(b) The plot panel. Each isotope gets its own simulated voltage spectrum on the requested scan range; dashed lines mark the individual hyperfine peaks.

Figure 4: Estimate output after a successful run.

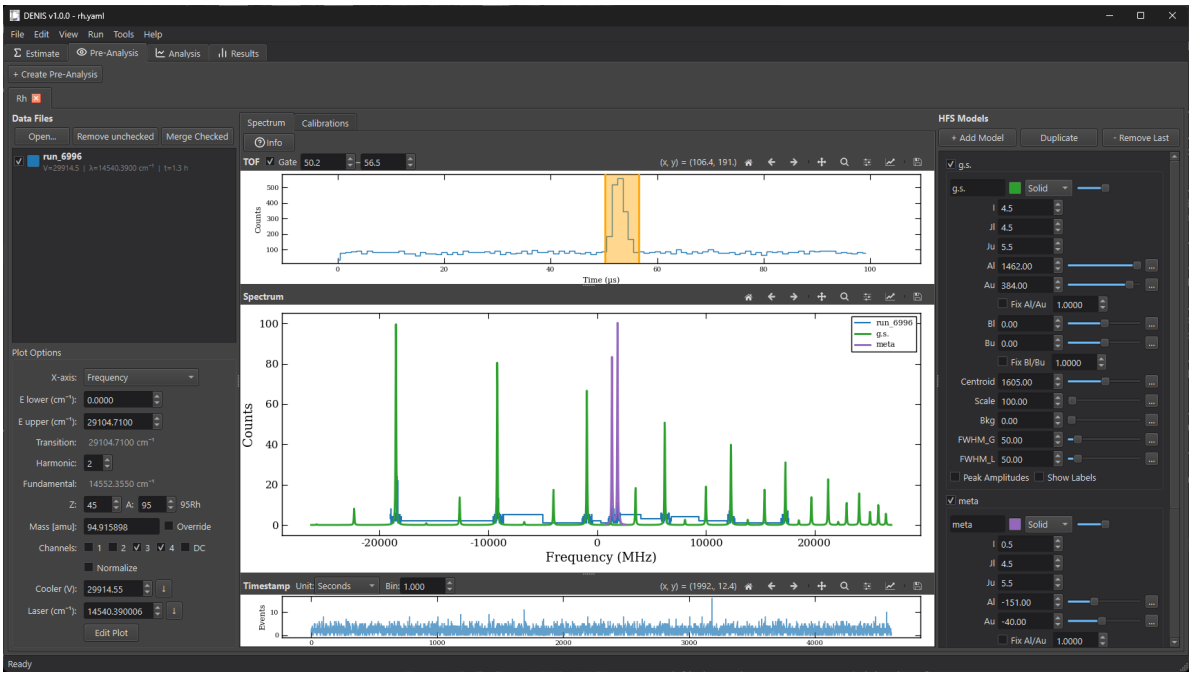


Figure 5: Pre-Analysis tab with one rhodium run loaded. Top centre: the time-of-flight spectrum with the gate highlighted in orange. Centre: the binned spectrum (blue data) with two HFS models overlaid (green ground state, purple isomer) on the right-panel parameters. Left: file list and binning controls; right: the HFS model parameter editor.

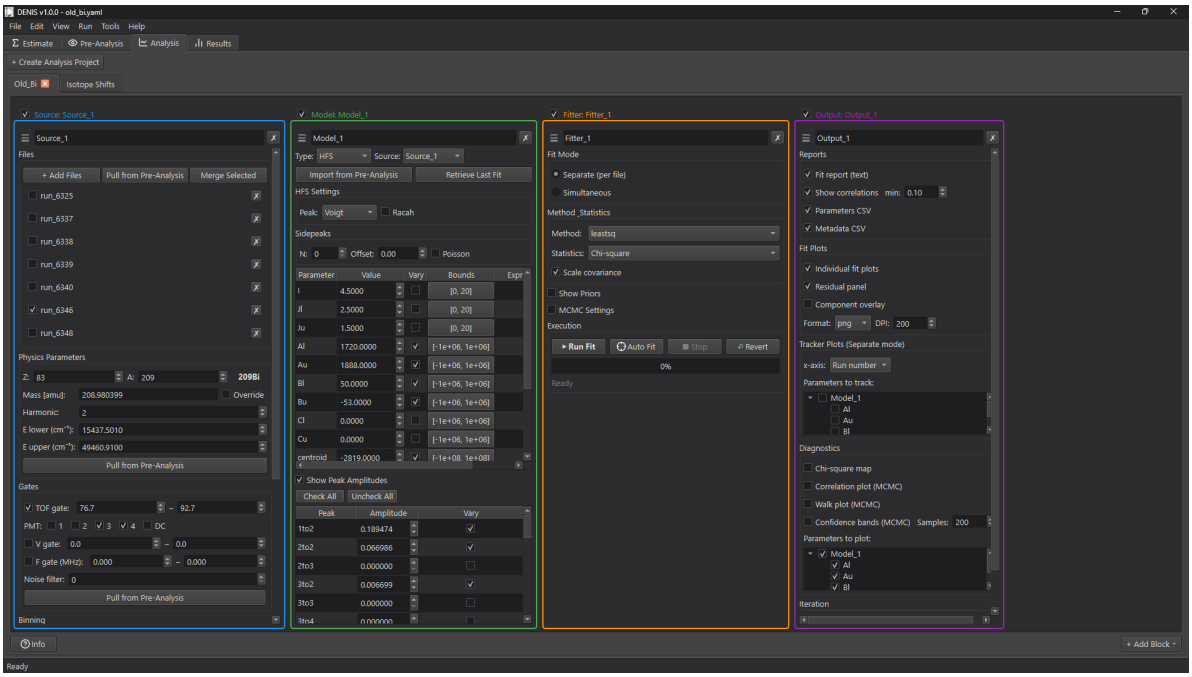


Figure 6: An **Analysis** project (here, an ^{209}Bi fit) showing the four-block pipeline from left to right: **Source** (file selection, physics parameters, ToF and binning), **Model** (HFS settings, parameter table with per-parameter modes, peak amplitudes), **Fitter** (method, statistics, execution controls), and **Output** (which reports, plots, and diagnostics to produce).

candidates against the data, and accepts the chosen result back into the model parameters in one undoable step.

3.5 Running the fit and reading the result

Click **Run Fit** on the **Fitter** block. The status bar reports progress; on completion DENIS pushes the fit results to the **Results** tab.

The **Results** tab lists each project in a tree, with one folder per iteration holding the fit report (reduced χ^2, Akaike and Bayesian information criteria), parameter tables, and the requested plots. **Edit Plot** opens a dialog for adjusting matplotlib styling (line widths, fonts, colours, axis limits); **Export** saves the plot in PNG, PDF, or SVG. Style adjustments are written next to the plot data as a `.style.json` so they survive re-loading the project.

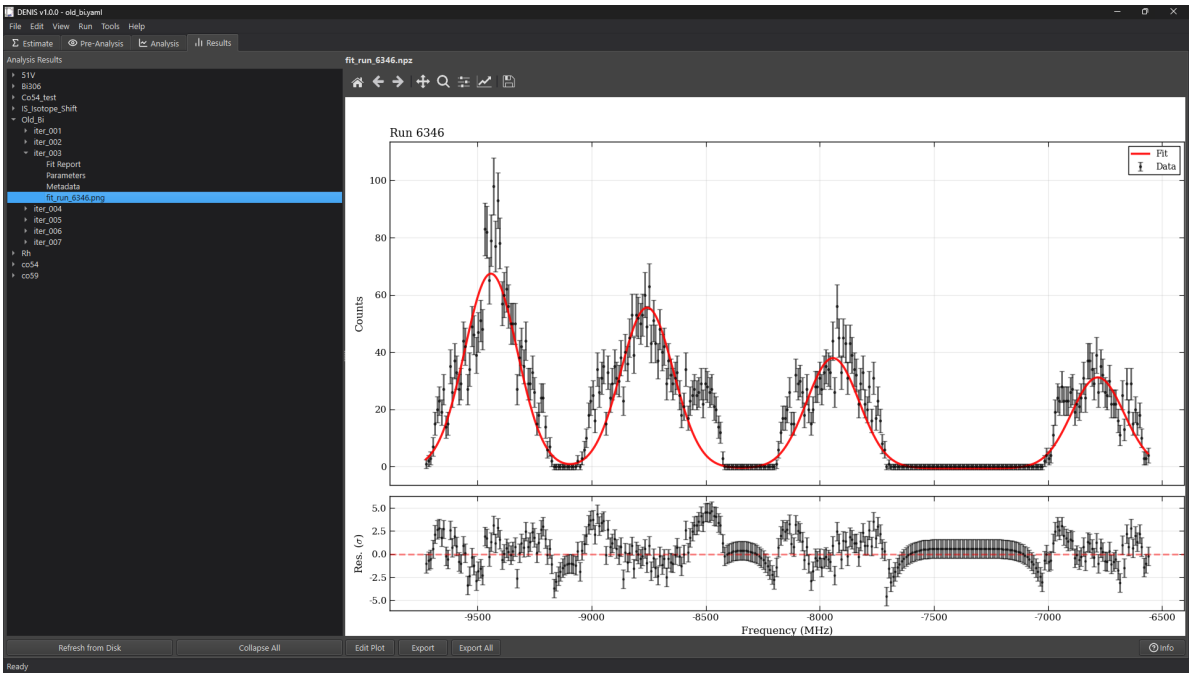


Figure 7: Results tab after a completed ^{209}Bi fit. Left: the project tree, with per-iteration sub-folders containing the fit report, parameters, metadata, and plot data. Right: the selected plot (data, model, residuals in σ) with the matplotlib navigation toolbar above and **Edit Plot** / **Export** controls along the bottom (not shown).

3.6 Combining isotopes (isotope shifts)

To compute isotope shifts across multiple analysis projects, switch back to the **Analysis** tab and open the permanent **Isotope Shifts** sub-tab. Click **Refresh from Projects** to populate one row per project, mark one isotope as the reference, then click **Compute Shifts**. The right-hand pane shows the combined plot, the shift table, and a log of how each shift was computed. **Export Table CSV** writes the shift table to disk and **Send to Results** pushes the figure to the **Results** tab.

The remaining sections of this manual document each tab in detail.

4 Estimate Tab

The **Estimate** tab predicts beam time before a measurement. Given an element, an electronic transition, an isotope chain, and beam parameters, it computes per-peak signal rates, the on-peak

time required to reach a target statistical significance, the number of voltage sweeps needed to cover the requested scan range, and a grand-total run time. A **Spectrum only** switch suppresses the timing pass and produces only the simulated lineshape — useful when planning the scan window itself.

The tab is a single surface with three resizable columns: the global parameters on the left (Figure 3), the isotope list in the middle, and a right column stacking the **Run Options** group, the plot display, and the streaming log (Figure 4). The expand button on the plot panel temporarily expands the plots over the whole tab; click it again to bring the parameter columns back.

4.1 Global parameters

The left column holds settings shared by every isotope.

Element & Transition. Element symbol, atomic number Z, the lower and upper levels of the electronic transition in cm^{-1}, and their total angular momenta J_l, J_u.

Beam. Acceleration voltage (kV), ion charge state, and laser-ion geometry — **collinear** or **anti-collinear**.

Laser. Harmonic order and the setpoint of the *fundamental* laser in cm^{-1}. As an alternative to a fixed setpoint, tick **Anchor to isotope**, choose an isotope and its state (g.s. or isomer), and enter the voltage offset **dV** from V_0 at which that resonance should sit; DENIS then back-solves the laser frequency required to place the centroid at that voltage.

Linewidth. Full-width-half-maximum used when drawing simulated spectra (MHz). Does not affect timing.

Scan. Voltage range (full window, ± half on either side of V_0), step size, and dwell time per step (ms).

The checkable **Statistics (Timing Mode)** group is the master switch for timing: unchecking it puts the whole tab in spectrum-only mode (rates, backgrounds, and yields are hidden, and the scan step size and dwell time are disabled). When checked it exposes:

- **Required** σ — target statistical significance per peak; defaults to 3σ.
- **Uniform timing (global default)** — when ticked, every measured peak gets the same on-peak time, computed for the strongest peak assuming full yield. Useful when a single dwell setting must cover the whole scan. Any isotope can override this locally with its own **Uniform timing** control.

4.2 Isotope entries

The middle column contains one expandable panel per isotope, added with the **+ Add Isotope** button at the top. Panels are reordered by dragging the hamburger handle and are color-striped by position so the same isotope reads consistently across the tab. Each panel exposes:

Mass number A and label used in plot legends and log headers.

Nuclear spin I integer or half-integer.

μ (μ_N) and Q (b) magnetic dipole and electric quadrupole moments.

Isotope shift (MHz) centroid offset from the reference isotope.

Yield (ions/s) expected ion rate (a scientific-notation field).

Efficiency one detected photon per $1/\eta$ ions; the log echoes this expanded as e.g. “1 photon per 500 ions”.

Peaks to measure how many of the strongest hyperfine peaks enter the timing estimate (default 1; raise it when several components are diagnostic).

Background either a flat rate (Hz), or a continuous rate combined with a TOF-gate width (µs) and the dwell time, which then collapses to an effective rate automatically.

Exactly one isotope is marked as the **Reference**. Its hyperfine A and B constants (entered in MHz, lower- and upper-state separately) are used to scale the constants of every other isotope via the Schmidt-moment ratio. Tick **Override A/B constants** on a non-reference isotope to bypass scaling and supply explicit values — useful when high-quality measured constants are already available (this is the case shown for ^{51}V in Figure 3).

Schmidt-moment auto-calculation. For an isotope whose μ is unknown, tick **Use Schmidt moment** to derive it from the shell model. The single-particle case takes a nucleon type (`p`/`n`) and an orbital (e.g. `f7/2`). For odd-odd nuclei, choose the two-particle variant and supply two orbital configurations together with the resulting spin I. The unquenched Schmidt μ is auto-filled into the panel's μ field (undoable with `Ctrl+Z`); the configured g_s quenching factor is applied during the run, and the log prints the configuration string next to the value actually used. The same calculator is available standalone under Tools → Schmidt Moment Calculator (Section 8).

Isomers. Tick **Has isomer** to expand a sub-panel for an isomeric state. The isomer carries its own spin, moments, isotope shift, and one of three yield specifications: an explicit **Yield** (independent of the ground state), a **Production ratio** in $[0, 1]$ that splits the parent's yield, or a **Spin distribution** σ that estimates the split from spin statistics — the latter is useful for surface-ion sources where high-spin states are populated preferentially. The isomer contributes its own peaks to the simulated spectrum and its own row to the timing table.

4.3 Running the estimation

Click **Run Estimation** in the **Run Options** group (right column). The log streams in real time and, on completion, contains in order:

1. A header summarising the inputs and, in anchor mode, the derived laser setpoint.
2. Per isotope: signal rate, on-peak time per measured peak, the number of sweeps required to accumulate enough statistics on each peak, and a subtotal in hours and in 8 h shifts.
3. A **Nuclear parameters** summary table with the μ values actually used (config or Schmidt) and the resulting A, B constants.
4. The grand-total run time across all isotopes.
5. Two peak lists — one sorted by intensity, one by voltage — covering every hyperfine component of every isotope and isomer.

The log is also written verbatim to a `.log` file in the run's output folder (`cls_<timestamp>` under the chosen **Output directory**); the YAML used for the run is preserved alongside it for reproducibility. The run additionally persists its plot arrays and peak table (`estimate_results.npz`, `peaks.csv`), and the save file records the run folder — so a saved session restores its plots, peak table, and log on load without re-running.

4.4 Simulated spectra

The plot panel shows a simulated voltage spectrum per isotope on the configured scan range, with vertical dashed lines at each hyperfine peak position (Figure 4b). The **View** drop-down switches between **Individual Spectra** (one panel per isotope), **Combined Overview** (all isotopes overlaid), and **Peak List** (a sortable table of every hyperfine component). **Edit Plot** opens the matplotlib style editor; **Open PDF** reveals the exported PDF in the system file viewer.

4.5 Loading a previous estimation

Load Estimation... (next to **Run Estimation**) reopens any previous `cls_<timestamp>` run folder for viewing — plots, peak table, and log are restored without re-running and without creating new output. Runs from before the `.npz` persistence existed hold only PDFs; their pages are rendered onto the live matplotlib canvas, so pan/zoom/save still work.

5 Pre-Analysis Tab

The **Pre-Analysis** tab loads measured data, gates the time-of-flight signal, bins the resulting spectrum on either acceleration voltage or Doppler-shifted rest-frame frequency, and overlays trial HFS models so the operator can read off initial parameters before the full fit.

Each pre-analysis is a sub-tab under **+ Create Pre-Analysis**; the sub-tab name (e.g. **Rh** in Figure 5) is shared with the Analysis tab when settings are pushed across (Section 6). The layout has two panes: a left column holding the **Data Files** list, the **Plot Options** group, and the **HFS Models** panel, and a wide centre pane with three plot sub-tabs — **Spectrum**, **Calibrations**, and **Cooler Voltage**.

Three view controls sit in the corner of the sub-tab row and apply to *every* open pre-analysis project at once: a **Layout** combo (**3 row stacked** = ToF / Spectrum / Timestamp as rows, or **2 row stacked** = Spectrum and ToF side by side with the Timestamp strip below, the original Data Viewer arrangement), **X** / **Y** grid toggles, and a sun-moon button that switches the plots to a *dark mode* (black canvas, white chrome, neon line colours). All three persist — the dark state both in the global settings and in the project save file.

5.1 Loading data

Click **Open...** to add one or more IGISOL-style ASDF run files; `clstools` [2] parses each one and a row is added to the **Data Files** list with a checkbox, an auto-assigned colour, and a short header showing the cooler voltage V, laser setpoint λ, and (after loading) the run duration. Tick the runs you want plotted; the panels redraw immediately. **Remove checked** drops the ticked rows, and the tri-state **Check all** box flips every row at once. A circular-arrow button re-reads a still-growing run from disk.

Default run colours cycle through the classic Windows 98 palette (navy, maroon, green, purple, ...), chosen for contrast between consecutively loaded runs. Every file (and every HFS model) carries *two* colours — one for the light canvas and one for dark mode — edited together in a dual-mode colour dialog (preset palettes, hex/`R,G,B` entry, and saved custom slots shared across the app). Clicking a row's colour swatch opens a small menu with the colour dialog, an opacity slider, and a line-style selector.

Right-clicking a run row offers **Remove**, **Check**/**Uncheck**, **Set centroid offset...** (a manual MHz shift consumed by frequency-domain merges), **Filter scans...** (Section 5.6), **Calibration...** and **Calibration overview...** (Section 5.11). Merged entries instead offer **View...**, **Edit...**, and **Export ASDF...**.

Different runs may sample different parts of the rest-frame spectrum even when the lab-frame voltage range is identical, because each run's cooler voltage and laser setpoint may differ. DENIS keeps each run's metadata so the rest-frame conversion is always applied per run, never globally.

5.2 The time-of-flight and timestamp gates

The TOF panel shows the time-resolved ion signal for the selected runs, histogrammed with an adjustable **Bin** size (default 1 µs, down to 1 ns for narrow bunches). Enable the **Gate** checkbox and set the window in µs by dragging the orange region on the plot or typing into the two value

boxes; only counts inside the window are histogrammed into the spectrum below, which re-bins live as the gate moves.

The Timestamp panel plots event counts over wall-clock time with a selectable **Unit** (Seconds / Minutes / Hours / Days) and its own **Bin** size. Its **Gate** restricts the spectrum to a chosen time window — the TOF and timestamp histograms themselves always show all events; only the spectrum is filtered.

The arrow-step convention is shared by the HFS-model value boxes. Clicking inside a value box places the cursor on a digit, and the up / down arrows then step the value by the order of magnitude of that digit — click a fine digit for small adjustments, a coarse one for big jumps. The active digit is highlighted.

5.3 Display axis and physics inputs

The **Plot Options** group configures the display. The **X-axis** drop-down offers five modes: **Voltage** (raw DAC scanning voltage), **Calibrated voltage** (mapped through the calibration table), **Calibrated beam energy** (cooler minus calibrated scanning voltage), **Wavenumber**, and **Frequency** (rest-frame MHz). The Doppler conversion uses the per-run cooler voltage, laser setpoint, transition energies (**E lower** / **E upper**, with derived **Transition** and **Fundamental** read-outs), the laser **Harmonic**, and the isotope mass — looked up from Z/A, with an **Override** tick to type a custom amu value. When the effective bin domain is frequency, the combo is restricted to **Wavenumber** and **Frequency**: a frequency-bin centre has no clean inverse back to volts.

Channels selects which PMTs (1–4, DC) to sum, **Normalize** scales each run to unit peak height, and the **Cooler (V)** / **Laser (cm^{-1})** fields override the per-run metadata for all files at once (a down-arrow button auto-fills from the selected file). The override is useful as a sanity check and for runs whose recorded values are known to be wrong.

5.4 Binning

The spectrum is binned *per scan step* by default: one bin per native DAC voltage step (or its Doppler-shifted frequency equivalent), which reproduces the legacy Data Viewer exactly and is structurally immune to the uniform-grid aliasing that produces doubled-count spikes when a fixed grid beats against the scan step spacing. The **Bin:** $N\times$ **step** spin in the Spectrum header groups N adjacent steps into one bin (count-weighted centres, summed counts) when more smoothing is wanted. Pre-Analysis shares the same binning engine as the Analysis Source block (Section 6), so a spectrum read here matches what the fitter sees.

5.5 HFS model overlay

The **HFS Models** panel (left column) adds trial hyperfine models. Each model is a checkable panel with a name, dual light/dark colours, a line-style selector, an opacity slider, and the parameter set: I, J_l, J_u (one row), then A_l, A_u, B_l, B_u, **Centroid**, **Scale**, **Bkg**, and Gaussian / Lorentzian FWHMs. **Fix Al/Au** and **Fix Bl/Bu** lock the pairs to a typed ratio — handy when the upper-state constant is poorly constrained; the locks carry over as fixed flags when the model is imported into the Analysis tab.

Each parameter row has a value box, a slider (with editable limits via the **...** button), and digit-aware arrow stepping. Tick **Show Labels** to annotate each component on the spectrum with its transition label, and **Peak Amplitudes** to expose a per-component editor: each peak can stay on its **Racah** intensity, be typed freely (**Free**), or be **Linked** to another peak by a ratio.

To overlay a ground state and an isomer, add two models and rename them (e.g. **g.s.** and **meta** in Figure 5). **Duplicate checked** copies ticked models; **Remove checked** removes them (undoable with `Ctrl+Z`).

5.6 Per-scan filtering

A *scan* is one full sweep of the voltage range. Right-click a run and choose **Filter scans...** to drop individual low-quality scans before binning: the dialog lists every scan with its start time, duration, event count, PMT count and rate, with bulk check/uncheck controls and an **Exclude rate** < threshold. Alternatively tick **Show scans** on the Timestamp panel (single non-merged run only), pan/zoom to a bad stretch, and click **Exclude scans in view**. Exclusions live in a registry shared with the Analysis tab — a filter set here applies to the same file there — and persist under the save file's top-level `scan_filters` key.

5.7 Virtual splits

Edit → Split File... turns one parent ASDF into two virtual sub-runs described by tiny `.vasdf` sidecar files, each carrying a raw-voltage gate plus its own cooler/laser metadata — useful when two isotopes were scanned in one run. Loading a `.vasdf` in Pre-Analysis or Analysis behaves like an ordinary run with a permanent voltage gate; the parent file is never modified.

5.8 Merging runs

When several runs measure the same isotope, tick at least two loaded, non-merged runs and click **Merge Checked**. The shared merge dialog (Section 6) asks whether to merge in the lab-frame **voltage** domain or the Doppler-shifted **frequency** domain — the default follows the current x-axis. Use voltage-domain merges when the runs share cooler voltage and laser setpoint, and frequency-domain merges otherwise (each file is shifted to the rest frame with its own metadata first; per-file centroid offsets and any GP centroid alignment are applied at the same stage). The result is a synthetic entry that bins, overlays models, and exports like an ordinary run.

5.9 Calibrations sub-tab

The **Calibrations** sub-tab shows three stacked diagnostics of the scanning-voltage DAC, plotted against the requested set voltage for every checked, non-split run: the readback response, the readback−set offset (supply offset and drift), and the step-to-step readback change (step uniformity). It is a quick health check before pulling a run into a fit; the per-run calibration *fit* and its overrides live in the dialogs described in Section 5.11.

5.10 Cooler Voltage sub-tab

The **Cooler Voltage** sub-tab diagnoses the stability of the cooler high voltage over each run using robust statistics (median reference V_{ref} and $\sigma = 1.4826 \cdot \mathrm{MAD}$, immune to spikes). A sortable per-run summary table reports V_{ref}, σ, peak-to-peak, spike count, signed extremes, and σ converted to its approximate Doppler impact in MHz. Below it, three panes show the raw per-event cooler voltage, the deviation from the run average (with $\pm\sigma$ / $\pm 3\sigma$ bands, a per-bin median drift line, spike dots, and a secondary $\approx \Delta\nu$ (MHz) axis), and the per-bin ripple amplitude (RMS and P95−P5). **Clip y to** $\pm 4\sigma$ keeps normal ripple readable when spikes are present; **Bins** sets the time binning. Virtual splits are skipped on both diagnostic sub-tabs — they share their parent's arrays.

5.11 Per-run voltage calibration

Each run carries a DAC→HV calibration table; a polynomial through it turns every event's DAC value into a real voltage, so a bad table biases the whole frequency axis (at 30 kV and mass 51, 1 V of calibration error is ∼18 MHz of centroid shift). DENIS fits this calibration itself on every load, so the numbers do not depend on which `clstools` build is installed, and by default *nothing is dropped*: a run whose calibration outliers would actually move its centroid gets a blinking **!**

badge on its row (in both Pre-Analysis and Analysis). Clicking it reports the outlier count, the centroid shift in MHz, and the axis tilt across the scan, with **Open calibration...** / **Keep as-is** — either way the blinking stops for good, and the acknowledgement is saved with the project.

Right-clicking a run and choosing **Calibration...** opens a dialog for the DAC→HV voltage calibration in force for that run. By default the app fits every calibration point itself; the override is a property of the file and is stored in the project. Under **Fit this run's own points** you may raise the polynomial order, exclude points (drop the first N settling points, an $n{\cdot}\sigma$ residual cut, or an explicit hand-picked set), or tick **Force zero intercept ($p_0 = 0$)** to constrain the calibration offset to zero for a chain known to have no DC offset — the offset is dropped from the fit, not fitted and then blanked, so the gain is the true through-origin slope. You can instead borrow another run's calibration or type the coefficients in directly. The dialog shows the resulting polynomial, the residuals, and the centroid shift the choice costs in MHz. Points are coloured by the order they were taken (a colour bar runs from first to last). Hovering a point reads out its set and readback voltages and its distance from both lines, in volts and in MHz at the run's beam energy, and picking a row of the point table rings its point. The residual panels are scaled to the points the line was fitted to: a dropped settling glitch sits on the panel's edge as a triangle instead of squashing the rest, and the right-hand axis gives the distance in MHz. The sun/moon button in the plot's header switches to a black canvas and is remembered; until it is set, it follows the Pre-Analysis dark-plot switch. **Calibration overview...** triages every loaded run at once, worst first. Overrides live in a process-wide registry shared by every tab and persist under the save file's top-level `calibrations:` key (acknowledgements under `calibration_acks:`), siblings of `scan_filters:`.

6 Analysis Tab

The **Analysis** tab is where measured data are fitted to a hyperfine model. Each fit lives in its own *analysis project* sub-tab, and a permanent **Isotope Shifts / Centroid Correction** sub-tab combines the fitted centroids across projects into shift tables and plots, and hosts the Gaussian-process drift correction.

6.1 Projects and the block pipeline

+ Create Analysis Project is a menu with two entries: **Sample Project** (the standard fit pipeline; the isotopes you actually want to measure) and **Reference Project** (identical pipeline, but its fitted centroids feed the Gaussian-process drift correction of Section 6.8 instead of the shift table). Right-click a project tab to convert between the two kinds, rename, or close it. A project chains four kinds of block, drawn left-to-right with a colour-coded border:

Source (blue). Loads the run files, applies gates, and bins to voltage or rest-frame frequency.

Model (green). Defines a satlas2 model (one HFS per isotope or isomer, plus optional sidepeaks and background/auxiliary models).

Fitter (orange). Picks the optimiser, statistic, sharing, constraints, and runs the fit.

Output (purple). Selects which reports, plots, and diagnostics are produced and pushed to the **Results** tab.

+ Add Block inserts a new block of any type; the hamburger handle on each block re-orders them by drag, the title checkbox disables a block, and dragging a block's right edge resizes it.

6.2 Source block

The **Files** list at the top of the source block holds the run files attached to this source. **+ Add Files** opens an ASDF/`.vasdf` file picker; **Import from Pre-Analysis** copies the ticked files, physics

parameters, gates, and cooler/laser overrides from a pre-analysis project in one step; **Merge Selected** produces a synthetic merged entry analogous to the merge in the **Pre-Analysis** tab.

Physics Parameters (Z, A, mass, harmonic, lower / upper levels) feed the per-run rest-frame conversion. The mass field shows the AME-table value; tick **Override** to enter a custom value.

Gates are the standard set: TOF window in microseconds, PMT channel mask plus an optional DC channel, lab-frame voltage gate, rest-frame frequency gate (always entered in MHz), and a noise-filter threshold (maximum events per timestamp; 0 disables).

Binning. **Bin mode** chooses the spectrum domain — **Frequency** (rest frame) or **Raw Voltage** (lab frame, one bin per unique DAC value). In frequency mode, **Bin definition** selects how the axis is cut: **Per scan step** (the default — one bin per native voltage step, Doppler-projected; structurally immune to uniform-grid aliasing; a measured step whose photons all fall outside the ToF/PMT gates stays as a zero-count bin, as in **Raw Voltage**), **Auto** (clstools' uniform grid at roughly one bin per step — can alias against the step spacing, producing doubled-count spikes), **Fixed bin count**, or **Fixed bin width [MHz]**. **Bin multiple** groups N adjacent native steps into one bin (per-step and raw-voltage binning). The y-error model (**Poisson** $\sqrt{y+1}$ default, with an exact Poisson interval substituted below 10 counts), the x-column, and an optional x-error from the per-step voltage spread complete the policy.

Right-clicking a run row offers per-file *binning overrides* (freeze different binning for one run only, copy an override to the checked runs, show the effective config), the binning diagnostics, **Filter scans...**, and **Calibration...** — the same scan-filter and calibration machinery as Pre-Analysis (Sections 5.6, 5.11), backed by the same shared registries.

Binning dialog. The **Binning** button opens a three-tab health check of the actual binning for every checked file. **Summary** is a per-run table with occupancy columns (empty bins with a scan-gap-aware denominator, median and total counts). **Run detail** shows one run's spectrum with its bin edges, an optional raw-event rug, a warning strip (fallbacks, width mismatches, empty-bin and aliasing flags), and a scale strip reporting the raw-voltage bin size, the measured 1 V $\approx N$ MHz conversion from the run's own events, and the equivalent rest-frame bin width. **Compare** re-bins all runs onto one grid and shows them as a heatmap.

6.3 Model block

A **Model** block builds one satlas2 model. **Type** offers **HFS** (default), **Voigt**, **Skewed Voigt**, **Exponential Decay**, **Piecewise Constant**, and **Polynomial** — combine an HFS model with a Polynomial on the same source for a sloping background. To seed parameters from the trial overlay used in pre-analysis, click **Import from Pre-Analysis**; **Retrieve Last Fit** re-imports the most recent fit values to refine on top of them.

HFS settings. The HFS lineshape is a Voigt profile (the satlas2 version in use supports only Voigt for HFS; use the dedicated **Skewed Voigt** model type for asymmetric single peaks). Tick **Racah** (default) to constrain peak amplitudes to their angular-momentum ratios; untick it and use the per-component editor under **Show Peak Amplitudes** when peaks deviate from Racah ratios — e.g. optical-pumping effects. **Sidepeaks** adds N charge-exchange satellites at a fixed **Offset**, optionally with Poisson-distributed amplitudes.

Parameters. The parameter table has five columns: **Parameter**, **Value**, **Bounds** (min, max), **Mode**, and **Expression**. There is no separate vary checkbox — the **Mode** combo is the single source of truth:

Free. Floats during the fit (the only varying mode).

Fixed. Held at its value. (I, J_l, J_u are always fixed — they are physics, not fit parameters.)

Equal / Ratio / Offset. Tie the parameter to another via an auto-generated expression.

Custom. A free-form expression typed in the **Expression** cell, validated live (red tint on errors) and again in a blocking pre-fit check that catches unknown names, self-references, and dependency cycles.

Model-block expressions use bare local names (`A1`, `centroid`, ...) and are lowered to one full `Run_<id>___<Model>___<param>` name per source at fit time; the Fitter block's **Advanced Constraints** use the full names directly.

6.4 Fitter block

Fit Mode chooses how sources combine:

Separate (per file) (default). One independent fit per run, in parallel across worker processes; each run gets its own parameter copy.

Simultaneous. One satlas2 fitter over all runs, and the **Parameter Sharing**, **Advanced Constraints**, and **Common Grid** groups appear. Sharing is *opt-in*: tick a bare parameter and choose **Model** (share across same-named models) or **All** (share across every source). Each share is an explicit tie, visible in the Expression column; a centroid tied to another source counts once in the isotope-shift averages. A **Common Grid** option re-bins all sources onto one shared x grid (width user-set or auto); each source keeps only the bins it measured.

Method selects the optimiser (`leastsq`, `least_squares`, `slsqp`, `cobyla`, `nelder`, `powell`, or the MCMC sampler `emcee`). **Statistics** selects the objective: **Chi-square**, **Gaussian LLH**, or **Poisson LLH** (recommended at low counts). **Scale covariance** (off by default) multiplies parameter errors by $\sqrt{\chi^2_{\mathrm{red}}}$; it is forced off for likelihood fits.

A likelihood needs a *scalar* minimiser, so `leastsq` and `least_squares` — which fit a residual *vector* — cannot be combined with **Poisson LLH** or **Gaussian LLH**. satlas2 does not refuse that pairing; it silently substitutes SLSQP, which on a hyperfine spectrum can diverge to a nonsense centroid while still reporting success. DENIS substitutes `nelder` instead, flags it under **Statistics** before the fit runs, and writes a `[Minimiser]` line at the top of the fit report; choose `nelder`, `powell` or `emcee` to decide for yourself. Their error bars are estimated numerically by lmfit as $2\times$ the inverse curvature at the minimum. That is right for **Chi-square**; under a likelihood they come out $\sqrt{2}$ (41 %) too large, because the factor 2 assumes a χ^2 while satlas2 minimises $-\ln L$, half of one. The Fitter block warns about it, and **Correct likelihood error bars** — off by default and shown only when it applies — divides them by $\sqrt{2}$; corrected, they agree with `emcee` to a few per cent. The report states which was done. The square **?** button beside **Method** opens a practical guide to all of these choices: which minimiser and statistics to use when, where each one's error bars come from, and how to set up `emcee`.

Show Priors reveals a Gaussian-prior table (full parameter names; each prior adds a penalty term) and **MCMC Settings** exposes walkers, steps, burn-in, thinning, and auto-convergence — both take effect in **Simultaneous** mode (priors and advanced constraints are ignored, with a pre-fit warning, in Separate mode).

Burn-in. The walkers start in a tight ball at the Model block's values and need a while to spread over the posterior. **Burn-in** discards those first steps of every walker; keeping them widens the error bars — by 10–40 % on a Yb spectrum, where the walkers settled after about 300 steps. Set it where the traces of the walk plot have settled (the plot marks it with a dashed line), typically 5–10 autocorrelation times τ, which the report lists under `[[Burn-in]]` once a burn-in is set; run enough steps for about $50\,\tau$ after it. The values, errors, correlations, corner plot, confidence band and plotted fit all use the steps after the cut, and with an adequate burn-in the

`emcee` errors of a well-behaved fit agree with `leastsq`. Starting `emcee` from a converged point fit shortens the burn-in; **Thin** only saves memory.

An `emcee` fit — like any Poisson/Gaussian LLH fit — reports its fit statistics (data points, χ^2, reduced χ^2, AIC, BIC) recomputed from the residuals of the plotted curve, and the report says so. The sampler's own numbers are not a χ^2: lmfit evaluates an `emcee` objective once, gets the summed log-likelihood as a single number and reports "1 data point" with $\chi^2 = (\ln L)^2$.

Run Fit executes; the progress bar blends per-run completion with in-run iteration progress. **Stop** cancels the fit: separate fits stop at once (their worker processes are ended), a simultaneous fit at its next optimiser iteration; **Run Fit** returns once the fit has stopped, and a cancelled fit's partial results are neither saved nor sent to Results. Closing the window or the project while a fit runs asks first and stops it. **Revert** restores the model parameters to their pre-fit snapshot. **Find Parameters** opens the Auto-Fitter (Section 6.6).

6.5 Output block

Reports produces the text fit report (with the correlation matrix above a **min** threshold), plus **Parameters CSV** and **Metadata CSV**. **Fit Plots** controls per-run figures: **Individual fit plots**, optional **ToF plots** (ToF only / ToF + spectrum / ToF + spectrum + fit), the **Residual panel**, **Hide zero-count bins**, **Component overlay** with per-component checkboxes, and **Fit values on plot** — an on-plot values box ("Name = value ± err") fed by a per-parameter picker tree, re-rendered verbatim by the Results tab. The **Plot Options...** button opens the per-plot-type styling dialog (global rcParams, fit-plot shade-under-fit and values-box size, tracker/walk/correlation/χ^2-map styles, and the isotope-shift plot).

Tracker Plots (Separate mode) plots chosen parameters against **Run number** or **Start time**. **Diagnostics** adds the χ^2 map (any method) and, for `emcee`, correlation / walk plots and confidence bands. The walk plot shows every step, with the burn-in cut as a dashed line; the correlation plot is a corner plot of the samples after the cut — filled 2-D credible regions at 0.5, 1, 1.5 and 2σ (11.8, 39.3, 67.5 and 86.5 % of the samples), step histograms with the 16/50/84 % lines, and each value in its panel title. Its style, colour scale and font are set under Settings > Correlation; **Style = Scatter** keeps the earlier rendering. **Iteration** numbers the output folder (**Auto** increments `iter_001`, ...; **Manual** uses a custom label).

Update iteration. The **Update iteration** button regenerates the *last* fit's iteration folder with the current output toggles, *without* re-running the fit — useful when you realise a plot was left unticked. Newly-ticked walk/correlation plots are rebuilt from the saved MCMC chains; only the χ^2 map and confidence bands need a true re-run (the dialog says so rather than failing silently).

Results are written to `<output_root>/analysis/<project>/iter_NNN/` — the config snapshot, report, CSVs (including a per-run binning summary and any binning warnings), `plots/` with `.npz` sidecars for live re-rendering, `tracking/`, `diagnostics/`, and `chains/` for MCMC — and forwarded to the **Results** tab.

6.6 Auto-Fitter

Find Parameters on the **Fitter** block opens the Auto-Fitter. It runs a multi-start parallel sweep under the Fitter block's own objective (`leastsq` for a chi-square fit, a scalar minimiser under a likelihood, so the sweep ranks its starts by the statistic the real fit will use) — **Starts** Gaussian perturbations of the current parameters (width set by **Spread**; the first start is always the unperturbed guess), each fitted in a worker process — and ranks the results by the objective that fit minimises (χ^2, or $-\ln L$ under a likelihood) in a live table with a fit preview; the reduced χ^2 shown beside it comes from the residuals, as in the fit report. **Refine** re-seeds a new round around the selected result with the spread halved; **Accept Selected** writes the chosen parameters back into the Model blocks as one undoable step. The Auto-Fitter only seeds initial guesses —

run the real fit afterwards. Fixed and expression-tied parameters are never perturbed, and in simultaneous mode the sweep respects the Fitter block's parameter sharing.

6.7 Isotope Shifts / Centroid Correction sub-tab

The permanent **Isotope Shifts / Centroid Correction** sub-tab combines fitted centroids into shift tables and a comparison plot. A **Reference Project** is listed automatically and marked as the shift reference, since being the reference is what the project kind means. The left panel holds a **Setup** tab and the **Reference Correction (GP)** tab (Section 6.8); the right panel holds **Plot**, **Table**, and **Log**.

Refresh from Projects populates one row per sample project with its label, mass number, run selection (**Weighted Average** — inverse-variance with Birge inflation when runs scatter — or a single run), centroid, and statistical error; the **Ref** radio marks the reference isotope. A collapsible **Run preview** panel under the table shows the highlighted run's spectrum, fit curve, and reduced χ^2. **Systematic Error** is either one manual MHz value or an auto-estimate from the run-to-run jitter of cooler voltage and laser setpoint (via the Doppler sensitivity).

Compute Shifts produces the plot and a 14-column table that splits every uncertainty into its budget: per-isotope σ_{fit} / σ_{scatter} / σ_{corr} (from the GP) / σ_{volt}, and per-shift statistical, scatter, correction, systematic, and total components. When two isotopes share the same GP corrector, the correlated part of the correction cancels in the shift and the cross-covariance is subtracted. $\delta\nu$ arrows are labelled in the compact `value(err) MHz` form. **Plot Options...** configures layout (stacked / overlaid), content, x-axis, centroid and reference lines, arrows, boxed labels, and colours. **Send to Results** writes the study (CSV + log + live plot) to an `IS_<study>` iteration and forwards it to the **Results** tab; **Export Table CSV** writes the table alone.

6.8 Reference correction (Gaussian process)

Slow drifts of the wavemeter or laser lock move every centroid measured over a campaign. DENIS implements the time-dependent reference-centroid correction of van den Borne (2025) [4]: fit the reference isotope's scans in one or more **Reference Projects**, then, on the **Reference Correction (GP)** tab, pick a kernel (**RBF + WhiteNoise**, **Matérn(5/2) + WhiteNoise**, or the **Composite** kernel with a periodic day/night term) and click **Fit GP**. Each fitted reference centroid becomes one training observation $(t, \text{centroid}, \sigma)$; the diagnostic plot shows the scatter, the MAP curve, and $1\sigma/2\sigma$ bands (optionally with an MCMC posterior for diagnostics — prediction always uses the MAP point; PyMC v5 is required).

Compute per-file corrections then fills a table with one row per sample run — fitted or not, since a correction needs only the run's start timestamp — carrying the correction $\mu_{\text{GP}}(t)$, its σ, and a $|\mu|/\sigma$ significance colouring. Each row's **Mode** is **Auto** (use the GP), **Manual** (type a value), or **Off**; the global **Apply correction at fit time** checkbox gates the whole mechanism. Corrections are subtracted from the binned MHz axis after the Doppler transform — consistently in the fit pipeline, the merge dialog, and the Auto-Fitter — and the propagated correction uncertainty (including the GP cross-covariance between runs) feeds the σ_{corr} columns of the shift table. The trained GP and the table round-trip through the save file; corrections saved without their GP are dropped on load with a prompt to re-fit.

The merge dialog's **Align centroids before merging** box is enabled only once that table holds a row for every file entering the merge; it moves each file onto the drift-free frame *before* they are histogrammed together, so the drift cannot smear the merged peak. The order is therefore fit the GP, compute the corrections, then merge. A merged entry gets no row of its own: it inherits its constituents' corrections, whose n-weighted mean and propagated σ are written into its fit metadata.

If PyTensor (PyMC's numerical backend) finds no C compiler on `PATH` it prints a `g++ not detected` warning and falls back to a pure-Python evaluation path. The result is identical; only

the runtime changes, by roughly one to two orders of magnitude. Installing MSVC Build Tools (Windows) or `m2w64-toolchain` restores the compiled path.

7 Results Tab

The **Results** tab is a viewer for everything produced by the **Analysis** tab: fit reports, parameter tables, metadata, and the plots requested in each project's **Output** block.

The left pane (Figure 7) is a tree keyed by analysis-project name. Each project node expands into one sub-folder per iteration of **Run Fit** (e.g. `iter_001`, `iter_002`, ...), and each iteration contains the artefacts written for that run: **Fit Report**, **Parameters**, **Metadata**, the binning summary and warnings, the saved plots, and free-text **Notes** nodes at both project and iteration level (autosaved as you type). A freshly finished iteration is highlighted in amber with a **(NEW)** marker until you open it. Selecting a leaf shows it in the right pane — a matplotlib canvas for plots, a table for CSVs, a read-only text view for reports.

Iterations arrive three ways: automatically when a fit finishes, restored from the save file on load (exactly the iterations the session recorded — missing folders are reported, nothing else is pulled in), or via **Refresh All** / **Refresh Project**, which re-scan the output directory for results produced outside the session.

Plots saved by DENIS include their underlying data as a `.npz` sidecar, so the canvas is a *live* matplotlib figure rather than a flat image — fit plots (with residual panel, component overlays, and the fit-values box), ToF plots, trackers, MCMC walk / correlation plots, χ^2 maps, and isotope-shift comparisons all re-render from data. Use the matplotlib toolbar to pan and zoom, and **Edit Plot** (or right-click the canvas → **Edit plot...**) to open the style editor.

7.1 The plot editor

The editor is modeless and applies live. Six tabs cover the figure (size, DPI, margins), the axes (title and labels with real font pickers, sizes, limits, linear/log scales, tick density, grid), the artists (per-artist colour, width, style, marker, alpha, position / rotation / z-order, font family + bold/italic for text, and a **Remove artist** button), an **Add** tab (text, annotations with arrows, vertical / horizontal lines and spans — placed by coordinates or by clicking the plot), the legend, and export. `Ctrl+Z` / `Ctrl+Y` undo and redo edits inside the editor. **Click to select / drag on plot** lets you pick artists by clicking and drag text labels and annotation arrows into place.

Closing the editor on a live `.npz` plot writes a `.style.json` next to the plot data; on the next render the style is re-applied automatically, so the figure re-opens the way you left it (deleted tagged annotations stay deleted). Right-click a plot row to **Copy plot style** and **Paste plot style** onto other plots — a checklist picks which property groups travel (figure size, title/label text, font sizes, limits, grid, label positions, artist styles, annotations), and a bulk entry pastes to every checked plot at once.

7.2 Exporting

Export writes the active plot in PNG, PDF, or SVG at 300 dpi with the current styling baked in (text and CSV items are copied as files). **Export All** copies the entire selected iteration directory — reports, CSVs, plots, `.npz` data and `.style.json` sidecars — verbatim to a destination folder. Right-click a plot row and choose **Export data as CSV...** to write the plot's underlying arrays (data, errors, model, residuals, dense fit curve) with column selection, an optional header row, a delimiter choice, and a precision setting.

7.3 Refresh, collapse, and deletion

Refresh All re-reads the whole output directory (including projects not open in the Analysis tab); **Refresh Project** does the same for the active project only; **Collapse All** folds the tree. Deletion is available two ways and *removes directories from disk*: right-click a project or iteration row for a confirmed **Delete...**, or tick checkboxes on any number of projects / iterations and press **Remove Selected** for a batch delete with a single confirmation. Notes are deleted together with their owner; there is no undo.

8 Tools Menu

The Tools menu collects supporting calculators that are useful during CLS planning and analysis but do not belong to the main four-tab workflow. Each tool opens in its own window and stays open across tab switches, so values can be cross-checked while editing the **Estimate** or **Pre-Analysis** forms.

8.1 Schmidt Moment Calculator

Tools → Schmidt Moment Calculator computes single-particle and two-particle Schmidt magnetic moments (Figure 8). Choose **Single particle** for odd-mass nuclei and supply the unpaired nucleon's type (`p`/`n`) and orbital (`1s1/2`, `1p3/2`, ..., `1f7/2`, etc.); choose **Two particles** for odd-odd nuclei and supply two orbital configurations together with the resulting coupled spin I. The g_s **quenching factor** (default 1.0) lets the user apply a literature quenching to the bare-nucleon spin g-factor. The result is the predicted μ in nuclear magnetons, the corresponding g-factor, and a one-line description of the configuration that is also written into the Estimate log when Schmidt moments are used there (Section 4).

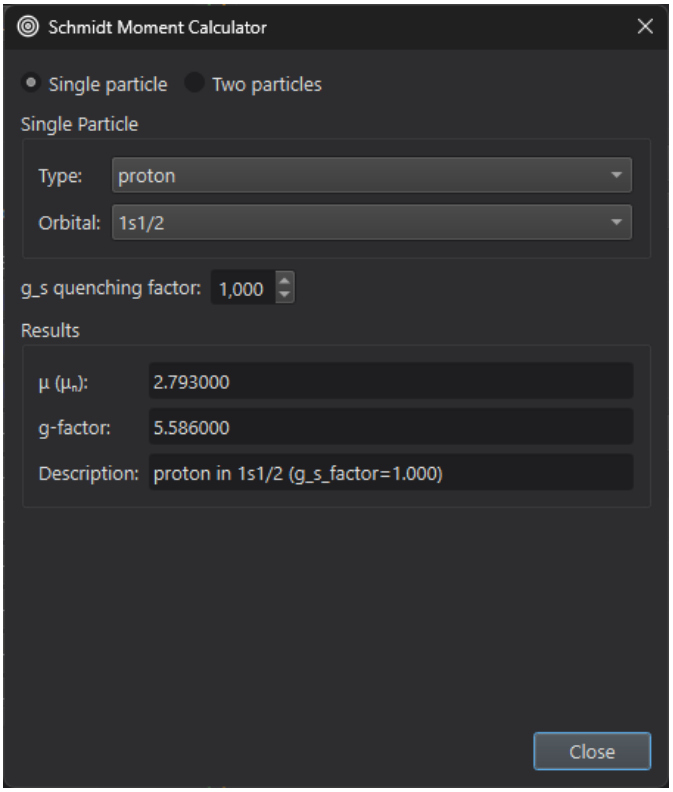


Figure 8: Schmidt Moment Calculator with a proton in the $1s_{1/2}$ orbital — the bare-nucleon prediction for a single odd proton.

8.2 Shell Configuration Plotter

Tools → Shell Configuration Plotter draws the nucleon shell-filling diagram for any nucleus (Figure 9). Enter Z and N in the top-left panel; the element symbol and mass number are derived automatically. The plot shows protons (red, left) and neutrons (blue, right) placed into spherical shell-model (jj-coupling) orbitals — not a deformed Nilsson diagram — with magic-number gaps marked between shells. The **Shell Model Results** block below the inputs prints the resulting proton and neutron configuration strings and the predicted ground-state J^π from the Nordheim coupling rule. **Plot Options** controls the depth of the shell ladder shown (**From shell** / **To shell**), labels, vacancy markers, n-quantum numbers, and circle size.

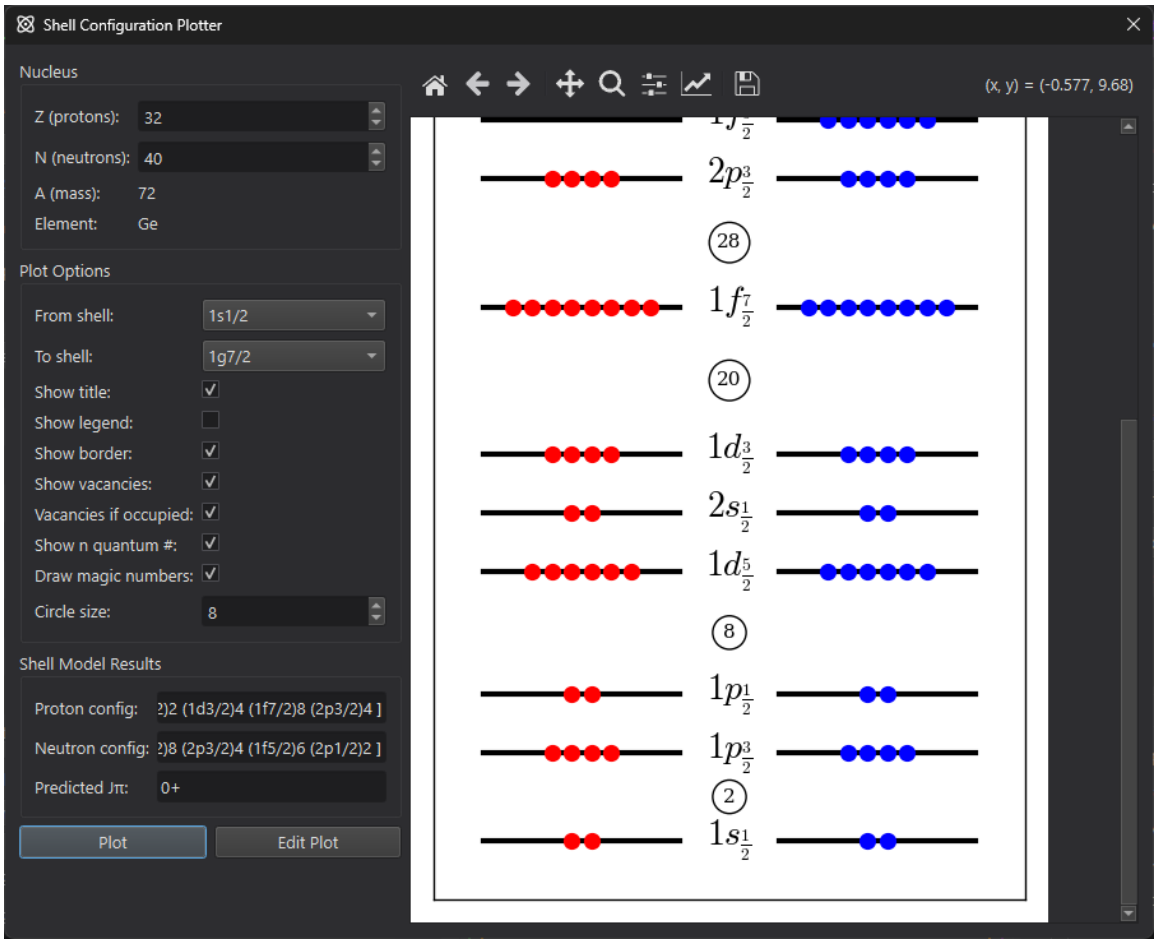


Figure 9: Shell Configuration Plotter for ^{72}Ge ($Z = 32$, $N = 40$). Both proton and neutron shells are filled through the $1f_{5/2}$ orbital; the predicted J^π is 0^+ as expected for an even-even nucleus.

8.3 Unit Converter

Tools → Unit Converter converts a laser-spectroscopy quantity into every other unit it is likely to be expressed in (Figure 10). Enter a value and pick the input unit (nm, cm^{-1}, MHz, GHz, THz, eV); the converted values appear immediately in all six units. Useful when toggling between the laser-control software (often nm), satlas2 parameters (MHz), and published transition tables (cm^{-1}).

8.4 SHG Crystal Angle Calculator

Tools → SHG Crystal Angle Calculator computes the Type-I (o+o → e) phase-matching angle for second-harmonic generation in nonlinear crystals, from Sellmeier dispersion (Figure 11).

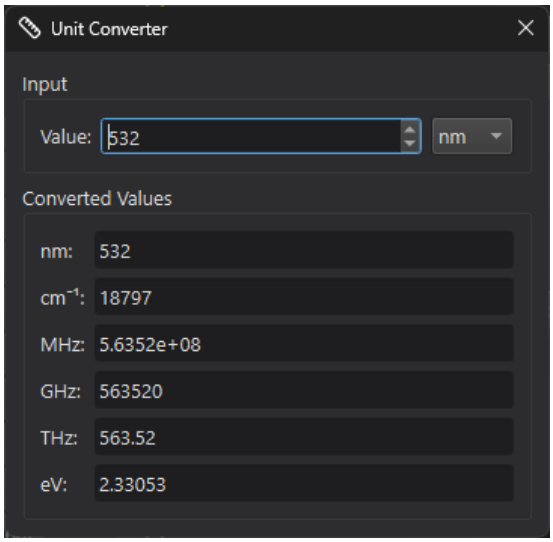


Figure 10: Unit Converter showing the second-harmonic of an Nd:YAG laser (532 nm) expressed in every supported unit.

Pick the crystal (**BBO**, **KDP**, **LBO**) and the calculation direction — **Wavelength** → **Angle** for typical use, or **Angle** → **Wavelength** for diagnostics on a fixed crystal mount. The result reports the phase-matching angle (or the matched fundamental) and the SHG wavelength; if the crystal cannot phase-match the requested line the result reads **No phase matching**. Only Type-I matching is implemented.

8.5 SFG / DFG Calculator

Tools → SFG / DFG Calculator combines two input wavelengths into a sum-frequency (SFG) or difference-frequency (DFG) output. Each input takes its own unit (nm, cm^{-1}, MHz, GHz, THz, eV), the combination is done in wavenumber space (additive in photon energy), and the generated wavelength is reported simultaneously in all six units. SHG is simply the SFG special case with two equal inputs.

8.6 Quick Plot

Tools → Quick Plot is a lightweight data viewer that lives outside the analysis pipeline (Figure 12). The **Load File** tab opens a delimited file (auto-detected or explicit delimiter) and lists its columns; **Enter Data** is a built-in spreadsheet that also accepts pasted tab/comma/semicolon/whitespace-separated data. Pick **X** and **Y** columns from the drop-downs, choose a chart type (**Line**, **Scatter**, **Step**, or **Bar**), and click **Plot**. Title, axis labels, grid and legend toggles, and a CSV / TSV save round out the tool; the standard matplotlib toolbar provides pan, zoom, and image export — useful for sanity-checking calibration files, ion-source trends, or any tabular data adjacent to a CLS measurement.

8.7 ASDF Viewer

Tools → ASDF Viewer... shows what is actually in a run file, as stored. Drop `.asdf` or `.vasdf` files on the window, use **Open...**, or right-click a run in the Pre-Analysis file list or in an Analysis Source block and choose **View ASDF...**; the viewer opens with that run selected (a `.vasdf` split opens its parent run). **Header** lists every non-array entry with its type; values are unconverted, and a note marks where DENIS reads one differently (`CoolerVoltage` is stored divided by 10 000). **Tables** shows the event table `raw`, one row per event, with columns named from `raw_header`, the calibration arrays side by side, and per-column minimum, maximum, mean and distinct values

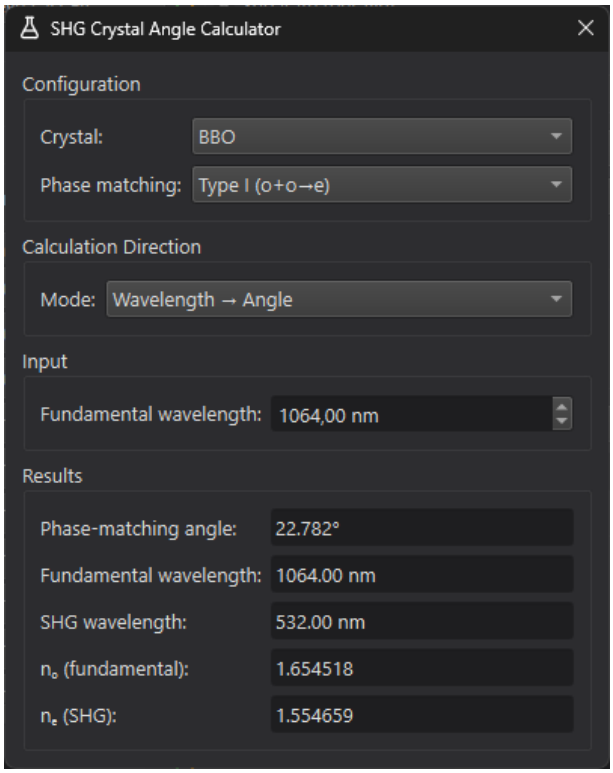


Figure 11: SHG Crystal Angle Calculator: BBO Type-I phase-matching angle (22.78°) for the second harmonic of 1064 nm.

(which PMT channels fired, the ToF range); a selection copies as tab-separated text and a table exports to CSV. **Tree** shows the complete ASDF tree. Hovering a column header or a header key explains the entry and its unit: `timestamp` is Unix time in seconds, `voltage` the scan-voltage setpoint in V, `time` the time of flight in µs, `channel` the PMT channel, `bunch_number` the bunch index over the whole run, and `cooler` the cooler voltage divided by 10 000. Files load in the background; a drop of 30 files or more is read on up to eight worker processes, about four times faster than one by one.

8.8 NIST ASD Browser

Tools → NIST ASD Browser… is a full in-app browser for the NIST Atomic Spectra Database, plus a multi-step laser excitation *scheme finder*. Enter a spectrum in NIST notation (e.g. `Si I`, `Ca II`) and click **Fetch**: data is loaded from a local cache when present, otherwise downloaded once from NIST and cached under `settings/nist_cache`, so subsequent sessions work offline. NIST data is fetched in vacuum; the **Air**/**Vacuum** toggle converts locally with the standard NIST dispersion formula.

Levels tab. Filterable level table (configuration, term, energy window, **Metastables only**) with three plot views: a Grotrian-style **Level diagram** (one column per configuration; bar opacity encodes the radiative lifetime $\tau = 1/\Sigma A_{ki}$, and hovering a level shows its energy, uncertainty, configuration/term/J, and τ), **Decay channels** of the selected level (one arrow per channel with λ and branching ratio — pick your detection channel here), and **Lifetimes vs energy** (metastable candidates jump out). **Add as starting level** sends the selected level to the Scheme Finder.

Lines tab. Filterable transition table (λ range, minimum A_{ki}, allowed / forbidden, involves-level) with a **Stick spectrum** (λ vs $\log_{10} A_{ki}$) and a **Line density** histogram (spectral congestion at a glance), plus manual axis-limit controls.

Scheme Finder tab. Searches 1–3-step excitation schemes from chosen (or auto-discovered ground + metastable) starting levels. Lasers are either named systems with a centre ± range and a role (**PUMP** / **PROBE** / **ANY**; each used at most once per scheme) or one broad wavelength window. Cuts on per-role A_{ki}, the minimum branching ratio of the watched fluorescence channel (denominator = all decay channels of the final level), a same/different-wavelength detection constraint, a probe orbital filter, and optional isobaric-contaminant checks (an issue is flagged when a scheme laser can excite the isobar *and* the isobar fluoresces near the detection wavelength). Results are scored ($\log_{10}$ weight $+\sum \log_{10} A_{ki} + \log_{10}\mathrm{BR} - 5$ per contamination issue), deduped by physical pathway, and ranked; selecting a row draws the excitation/detection diagram and a full text description, and **Export results CSV...** writes the ranked list.

The browser's state — spectrum, filters, search configuration, and the ranked schemes (self-contained, so they restore offline) — persists in the DENIS save file under the `nist_asd` key.

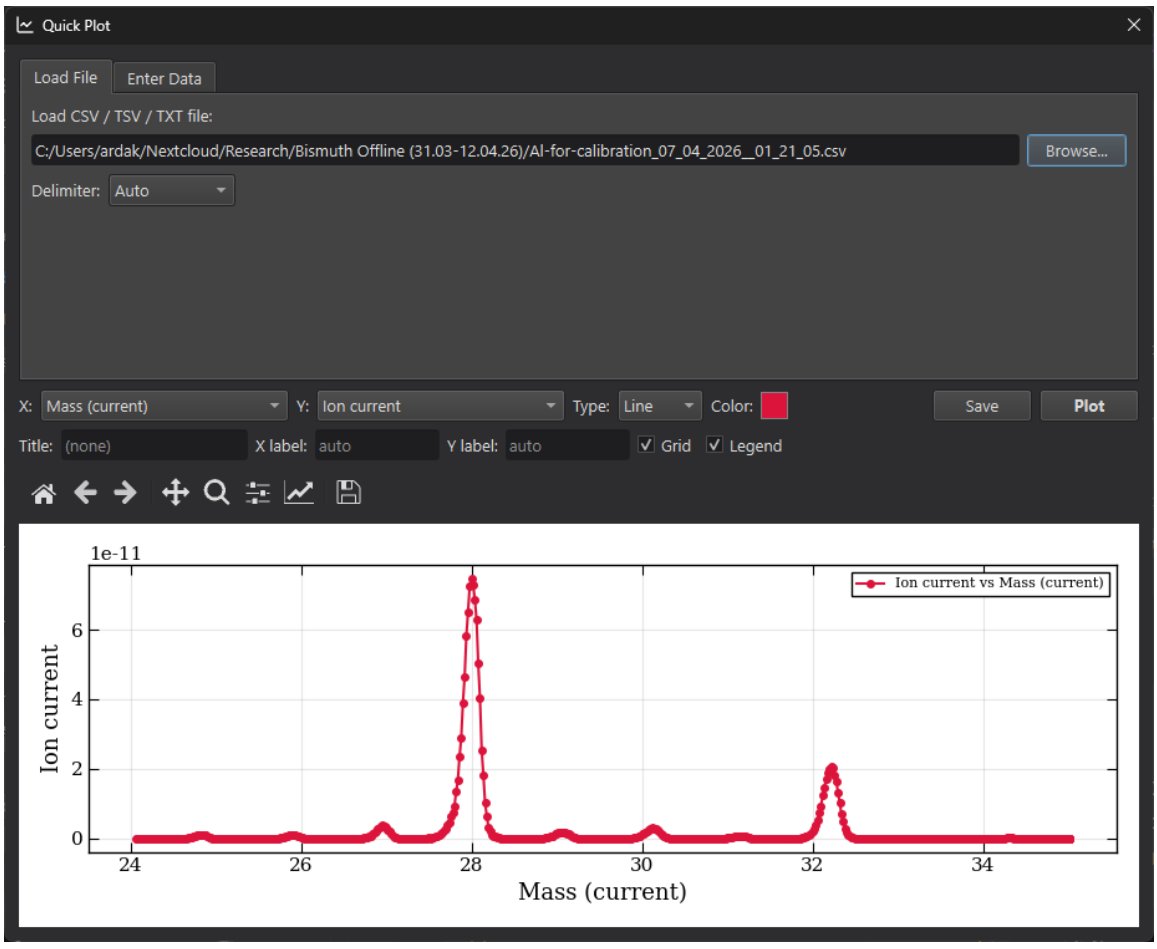


Figure 12: Quick Plot showing an ion-source mass-scan calibration loaded from CSV: ion current as a function of mass setting, with the sample isotopes appearing as well-resolved peaks near $A = 28$ and $A = 32$.

9 Settings and Save / Load

9.1 Settings dialog

File → Settings... opens a scrollable dialog of five groups covering the application-wide preferences.

General. Settings file location (with an **Open** button to reveal the folder), auto-conversion of file paths between Linux and Windows on save-load round trips, the **Theme** selector — **Dark (default)** or **Classic 98 (dark)**, a retro look with Windows-98-style 3D bevels, sharp corners, and a pixel font, applied live on OK — the UI scale factor for HiDPI screens (restart required), and the **Verbose session log** toggle. A session log is *always* written to a dated file under `logs/` (the newest 20 are kept); the toggle raises the log level to DEBUG on the next launch.

Performance. Maximum number of CPU cores the fitter and the Auto-Fitter may use in parallel (default: cores minus two).

Output. Default output directory, iteration mode (**Auto** numbers `iter_NNN` folders automatically; **Manual** uses a custom label), default plot format (PNG / PDF / SVG), and default plot DPI.

Fitting Defaults. Default optimiser, statistic, y-error model, and x-column for new Analysis blocks; whether the parameter-correlation matrix is shown by default. These seed new blocks only — existing projects keep their settings.

Plot Defaults. A **Global** tab sets matplotlib rcParams (line and marker sizes, font and tick sizes, axis line width, errorbar cap size, figure and save DPIs); per-plot-type sub-tabs (**Fit Plot**, **Walk Plot**, **Correlation**, **Chi-sq Map**, **Tracker**, **Preview / TOF**) override the global values for that plot kind — the fit-plot tab includes count-adaptive marker sizing and the shade-under-fit / values-box options, and the isotope-shift plot has its own tab in the Output block's **Plot Options...** dialog. **Reset All Plot Defaults** returns every value to the built-in defaults.

Settings are persisted to `settings/settings.yaml` inside the application folder (not an OS config directory), also reachable from the **Open** button in the **General** group. Older locations are migrated automatically on first run.

9.2 Save and Load

DENIS uses a single YAML format for all save / load operations. The file has one section per major tab plus shared top-level registries:

```
estimate: {...} # incl. last_run (restores plots/log on load)
preanalysis: {...} # projects: files, gates, models, plot options
analysis: {...} # projects (block pipelines) + isotope_shifts
scan_filters: {...} # per-scan exclusions, shared across tabs
calibrations: {...} # per-file voltage-calibration overrides
calibration_acks: [...] # acknowledged calibration warnings
nist_asd: {...} # NIST ASD Browser state and ranked schemes
```

The File menu follows the standard editor model, backed by dirty tracking (a fingerprint of the full save state):

Save (`Ctrl+S`). Writes to the currently loaded file with no dialog; falls through to Save As when no file is loaded yet.

Save As... (`Ctrl+Shift+S`). Always opens a file dialog and writes every tab.

Save Tab As.... Writes only the active tab's section, merging into an existing file so the other sections are preserved.

Load All... (`Ctrl+Shift+O`) / Load Tab... (`Ctrl+O`). Populate every recognised section, or only the active tab's. The status bar reports which sections were loaded. Estimate runs and Analysis iterations recorded in the file are restored into the Results tab without re-running.

New Window. Opens a second, fully independent DENIS instance (it skips the single-instance guard and the splash screen).

A legacy top-level Estimate format (with `element`, `transition`, `isotopes...` at the root) is detected automatically and routed into the **Estimate** tab so older configuration files continue to work. If a loaded file references data files that no longer exist, a locate-or-skip flow runs first and rewrites relocated paths back into the YAML.

On exit, DENIS prompts only when something actually changed since the last save or load. The prompt offers **Save** (to the current file, or Save As when none is loaded), **Don't Save**, and **Cancel**; **Cancel** is the default so an accidental `Enter` never discards work. An untouched or freshly-saved session closes silently.

A Physics reference

This appendix collects the formulae implemented in the `cls_estimations/` backend and used throughout the workflow. It is intentionally compact; for derivations and detailed discussion, see SATLAS2 [3] and the Handbook of Nuclear Physics chapter on nuclear spins and moments [1].

A.1 Doppler shift

For an ion accelerated through a potential difference V with charge state q and rest mass m, the relativistic Lorentz factor is

$$\gamma = 1 + \frac{qV}{mc^2}, \qquad \beta = \sqrt{1 - \gamma^{-2}}. \tag{1}$$

The lab-frame laser frequency ν_{lab} relates to the frequency seen by the ion in its rest frame, ν_{rest}, by

$$\nu_{\mathrm{rest}} = \gamma\,\nu_{\mathrm{lab}}\,(1 - \beta\,\hat{n}\cdot\hat{k}) = \gamma\,\nu_{\mathrm{lab}}\,(1 \mp \beta), \tag{2}$$

where the upper (lower) sign in the right-most expression corresponds to anti-collinear (collinear) geometry. Equation (2) is what DENIS inverts when the user specifies a target lab-frame voltage offset **dV** from V_0 for a chosen isotope — the laser frequency ν_{lab} is back-solved so that the desired rest-frame centroid sits at $V = V_0 - \mathrm{dV}$.

A.2 Hyperfine structure

For a level with electronic angular momentum J and nuclear spin I, the total angular momentum $F = I + J$ takes values $|I - J|, |I - J| + 1, \ldots, I + J$. The hyperfine energy of the substate F is

$$\Delta E_F = \frac{A}{2}\,C + B\,\frac{\frac{3}{2}\,C(C+1) - 2I(I+1)\,J(J+1)}{2I(2I-1)\,J(2J-1)}, \tag{3}$$

with

$$C = F(F+1) - I(I+1) - J(J+1). \tag{4}$$

A is the magnetic-dipole coupling constant and B the electric-quadrupole coupling constant; the second term in (3) vanishes when $I < 1$ or $J < 1$. DENIS uses (3) for both lower and upper electronic states; the observed transition frequencies are differences between the two. Selection rules are $\Delta F = 0, \pm 1$ with $F = 0 \to F = 0$ forbidden.

A.3 Schmidt magnetic moments

For a single odd nucleon in an orbital with quantum numbers ℓ, j the Schmidt prediction is

$$\mu = j\,g_j\,\mu_N, \qquad g_j = \begin{cases} g_\ell + (g_s - g_\ell)/(2\ell+1), & j = \ell + 1/2 \\ g_\ell - (g_s - g_\ell)\,\dfrac{j+1}{2j+1}, & j = \ell - 1/2, \end{cases} \tag{5}$$

with bare-nucleon g-factors $g_\ell^{(p)} = 1$, $g_\ell^{(n)} = 0$, $g_s^{(p)} = +5.586$, $g_s^{(n)} = -3.826$. The optional g_s quenching factor in the calculator multiplies the spin g-factor to account for in-medium effects. The two-particle case couples the two single-particle moments through the Lande formula at a chosen total spin I.

A.4 Fit objectives

The **Fitter** block exposes three statistics:

Chi-square. $\chi^2 = \sum_i \left(\frac{y_i - m_i(\theta)}{\sigma_i}\right)^2$, with σ_i taken from the **yerr** model ($\sqrt{y_i}$, $\sqrt{y_i + 1}$, or model-based). Suitable for bins with at least ~ 10 counts.

Poisson LLH. $-2 \ln L = 2 \sum_i \left[m_i(\theta) - y_i + y_i \ln(y_i / m_i(\theta))\right]$ for $y_i > 0$. Use this when the per-bin counts are low and the Gaussian approximation breaks down.

Gaussian LLH. Same kernel as χ^2 but with σ_i allowed to depend on the model. Useful when the noise model is heteroskedastic.

Scale covariance on the **Fitter** block multiplies the post-fit covariance matrix by the reduced χ^2, recovering the standard errors when the input σ_i are known to be mis-scaled.

B Troubleshooting

Shortcut launches but the estimation never finishes. Earlier versions froze when run from the Windows shortcut because the `pythonw.exe` process inherits `sys.stdout = None`, and the worker subprocesses crashed in `tqdm`. Both the worker thread and the per-run subprocess now redirect a missing `stdout`/`stderr` to `os.devnull` on entry; if you still see hangs from the shortcut, check that you are running a recent enough build and report the issue with the contents of `logs/denis_*.log`.

macOS layout looks white / clipped. Old releases did not pin the Qt style and macOS rendered the UI with the native Aqua style, which centres tabs, paints a white background under the dark-themed accent colours, and clips widgets sized for the Fusion style. Recent releases force `app.setStyle("Fusion")` and a custom dark `QPalette` in `main_window.py`; if the layout still looks off, raise the UI scale factor under File → Settings → General.

`clstools` fetch fails on `uv sync`. `clstools` is pulled from GitHub. If the host has no GitHub access, clone the repo locally and replace the entry in `[tool.uv.sources]` of `pyproject.toml` with a local path:

```
[tool.uv.sources]
clstools = { path = "../cls_tools" }
```

Then re-run `uv sync`.

Auto-Fitter never returns. The Auto-Fitter spawns one worker process per CPU core granted in the **Performance** settings (default `n_cores - 2`). If your machine is busy with another job, lower this number. The progress bar in the Auto-Fitter window also surfaces stalled workers — if a single core sits at 0 % for several minutes, click **Stop**, drop the **N starts** value, and restart.

Model overlay disappears when switching the Pre-Analysis x-axis to frequency. The conversion from voltage to rest-frame frequency requires valid laser setpoint, isotope mass, and cooler voltage. If any of these is missing in the source ASDF metadata, the axis silently falls back to MHz with no Doppler offset. Confirm that the **Cooler V** and **Laser cm^{-1}** fields in **Plot Options** are non-zero; override them manually if the run file is incomplete.

`Ctrl+C` / `Ctrl+V` / `Ctrl+Z` do nothing in a spinbox. The application installs a global event filter for clipboard and undo shortcuts. If a fresh shortcut still does not respond, restart the application; the filter is attached once during startup and a partial UI build can leave it out of date.

Decimal separator is shown as comma instead of dot. Every numeric spinbox in DENIS is created with a US-English `QLocale`, so the decimal separator is a dot regardless of the operating-system locale. If a field still shows a comma it was created outside the shared helper — report it as a bug with the field's name.

Capturing a crash. Every session writes a log to `logs/denis_<timestamp>.log` next to the application (a verbatim copy of the terminal output plus a structured log; the newest 20 files are kept). An unhandled exception is written under a clearly delimited `CRASH:` banner, and the startup banner records the versions needed for a bug report. Tick **Verbose session log** in File → Settings → General and restart to raise the log level to DEBUG.

C Citation and license

C.1 Citing DENIS

DENIS is developed in the open at github.com/ardakayaalp/DENIS and every release is archived on Zenodo. The version DOI for release 1.1.1 is 10.5281/zenodo.23497223 (release 1.1.0: 10.5281/zenodo.22994563; release 1.0.0: 10.5281/zenodo.22081267); the concept DOI 10.5281/zenodo.22081266 always resolves to the latest version. The canonical citation entry, including machine-readable metadata, lives in the `CITATION.cff` file at the repository root and is surfaced as a "Cite this repository" button on the GitHub project page.

Suggested citation.

> A. Kayaalp, *DENIS: Doppler Estimation and Numerical Inference for Spectroscopy*, version 1.1.1, Zenodo (2026). DOI: 10.5281/zenodo.23497223.

Acknowledged dependencies. DENIS relies on satlas2 [3] for hyperfine-structure fitting and on `clstools` [2] for IGISOL-style ASDF I/O. Please cite these packages alongside DENIS when reporting analyses produced with the toolkit. For background on the underlying technique, the Handbook of Nuclear Physics chapter on nuclear spins and moments [1] is a standard reference.

C.2 License

DENIS is released under the **MIT License** — see the `LICENSE` file at the repository root. You may use, modify, and redistribute it freely, provided the copyright and permission notice are retained. The bundled Lucide icons are MIT/ISC licensed; `satlas2` and `clstools` carry their own licenses.

C.3 Contact

Bug reports, feature requests, and contributions are welcome via the GitHub issue tracker. For correspondence about the underlying physics or for collaboration enquiries, contact the developer at arda.kayaalp@kuleuven.be.

References

[1] R. P. de Groote and G. Neyens. “Spins and Electromagnetic Moments of Nuclei”. In: *Handbook of Nuclear Physics*. Ed. by I. Tanihata, H. Toki, and T. Kajino. Living reference work chapter. Springer Nature Singapore, 2022, pp. 1–36. ISBN: 978-981-15-8818-1. DOI: 10.1007/978-981-15-8818-1_42-1.

[2] Vining, Andrea et al. *cls_tools: ASDF I/O for collinear laser spectroscopy data*. Software, accessed via GitHub. 2024. URL: https://github.com/andry3vi/cls_tools.

[3] W. Gins, R. P. de Groote, et al. “SATLAS2: an updated Python package for hyperfine structure analysis”. In: *(see project page)* (2024). TODO: replace with the latest published reference (DOI / preprint) for SATLAS2. URL: https://iks-nm.github.io/satlas2/index.html.

[4] B. van den Borne. “Investigation of the nuclear structure of silver isotopes via laser spectroscopy”. TODO: confirm the exact thesis title and add a persistent link. PhD thesis. KU Leuven, 2025.